Airbnb New User Bookings

Reza Almassi & Jad Mahmoud

University of Pisa — Data Mining & ML Project

Academic Year 2024/2025

Agenda

This part of the presentation explains the project from the beginning:

- What is the problem and why it matters
- What data we have (users + sessions)
- What we discovered in EDA (important patterns)
- How we cleaned the data (missing values, age, categories, dates)
- How we built features from sessions (aggregation)
- How we merged everything into a final matrix for modeling

Context and Motivation

Online travel platforms collect a lot of user data. Airbnb can use this data to **personalize** the experience for new users.

Why prediction helps:

- If we can guess a user's first booking destination, we can show better recommendations early.
- Better personalization can increase bookings and user satisfaction.

This project is based on the Kaggle competition: *Airbnb New User Bookings*.

Problem Statement (Very Simple)

We are given information about a **new Airbnb user**, such as:

- demographics (age, gender, language)
- signup info (app, method, traffic source)
- browsing sessions (clickstream actions)

Goal: Predict the country of the first booking destination.

There are **12 possible classes**: `NDF`, `US`, `other`, `FR`, `IT`, `GB`, `ES`, `CA`, `DE`, `NL`, `AU`, `PT`.

Main Challenges

This problem is not easy because:

- **Strong class imbalance:** most users are `NDF` or `US`
- **Missing values:** especially in sessions and age
- **Mixed feature types:** dates, categories, numeric values
- **Ranking metric:** Kaggle uses **NDCG@5** (it rewards correct destination being in top-5)

Project Objectives

We want to build a full pipeline:

1. Explore the data (EDA) to understand patterns
2. Clean the data carefully (fix missing values, wrong ages, etc.)
3. Create useful features (especially from sessions)
4. Merge everything into one model-ready dataset
5. (Next part of the full talk: modeling + hierarchy + ensembles)

Data Source and Files

The dataset comes from Kaggle and contains three CSV files:

- `train_users_2.csv`: 213,451 labeled users
- `test_users.csv`: 62,096 unlabeled users (for submission)
- `sessions.csv`: 10,567,737 session action rows

Important: Sessions do not exist for all users (only about **49%** of total users appear in sessions).

Files Overview (Table)

File	Rows	Columns	Description
`train_users_2.csv`	213,451	16	Labeled user records
`test_users.csv`	62,096	15	Unlabeled user records
`sessions.csv`	10,567,737	6	Clickstream actions

Users Table: What Columns Mean

Users table contains:

- IDs, dates (`date_account_created`, `timestamp_first_active`)
- demographic (`gender`, `age`, `language`)
- signup info (`signup_method`, `signup_app`, `signup_flow`)
- marketing info (`affiliate_channel`, `affiliate_provider`, `first_affiliate_tracked`)
- device/browser (`first_device_type`, `first_browser`)

Target (train only): `country_destination`.

Sessions Table: What Columns Mean

Sessions table records raw actions:

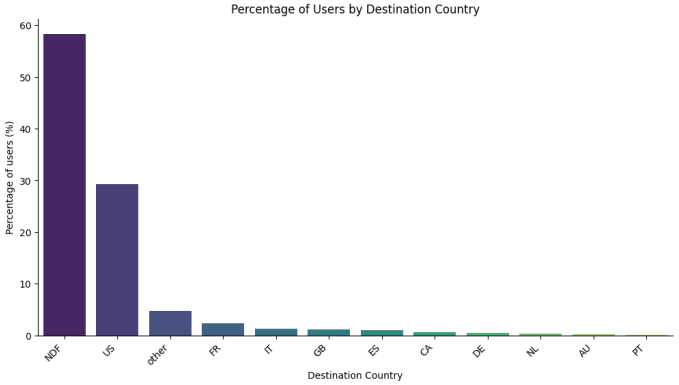
- `user_id`: links to user
- `action`: what page/function was used
- `action_type`: general category (click, view, data, submit, ...)
- `action_detail`: more detailed action description
- `device_type`: device used for that action
- `secs_elapsed`: time spent on that action

We later aggregate sessions into **one row per user**.

Target Distribution (Imbalance)

The destination labels are highly imbalanced:

- NDF = 58.35%
- US = 29.22%
- All other 10 classes together = about 12.43%



Why This Imbalance Matters

If a model always predicts `NDF`, it can still get **high accuracy**. But it would be useless for real destination prediction.

So we need:

- good preprocessing
- strong feature engineering
- models that can handle imbalance and multi-class ranking

EDA Goals

Before modeling, we explore the data to answer:

- Which features look noisy or wrong?
- What patterns relate to booking destinations?
- How do users behave in sessions?

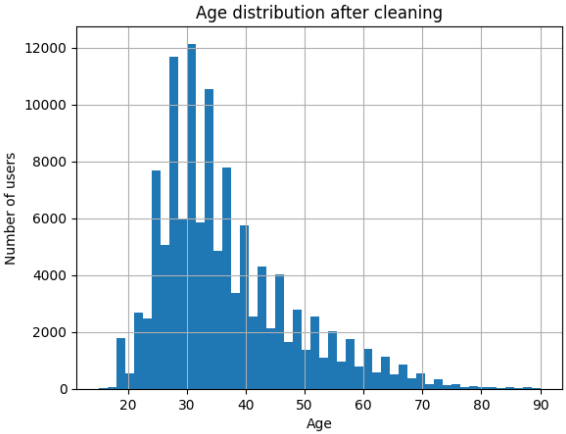
EDA: Age Distribution (Problems + Shape)

Age is important, but the raw column has issues:

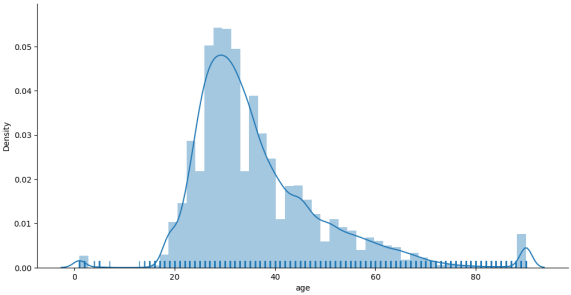
- some people typed birth year (e.g., 1988) instead of age
- some ages are too small (like 1) or too large (like 150)
- many missing values (about 42% missing after merge)

After cleaning, most users are between **25 and 40**.

EDA: Age Distribution (Plots)



figureDistribution Plot



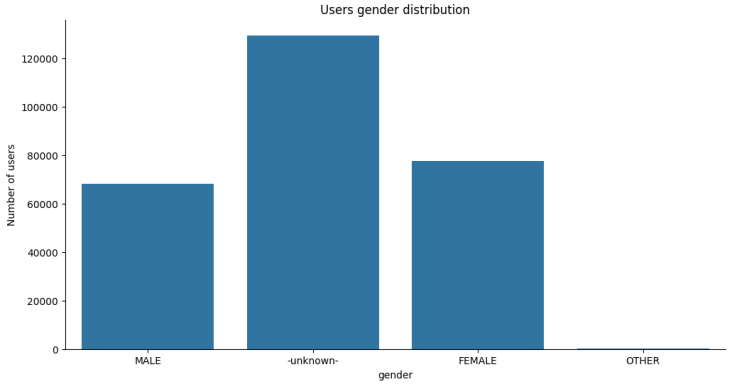
figureHistogram

EDA: Gender Distribution

Gender has 4 values: `MALE`, `FEMALE`, `OTHER`, `-unknown-`.

Key point:

- Around **45%** of users did not provide gender.
- Among known genders, male and female are close to balanced.



EDA: Preferred Language

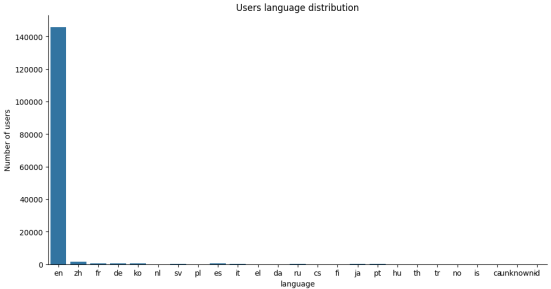
English is the dominant language overall.

When we exclude US bookings, we see more diversity:

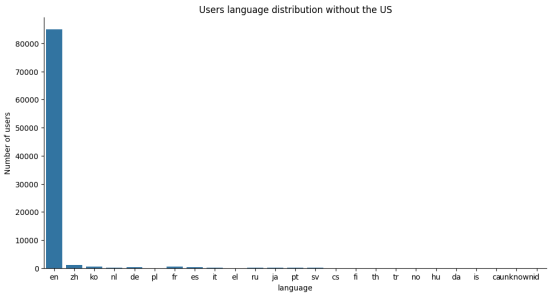
- Chinese (zh) becomes the second most common language
- French and Spanish also increase

EDA: Language Plots

Full Distribution

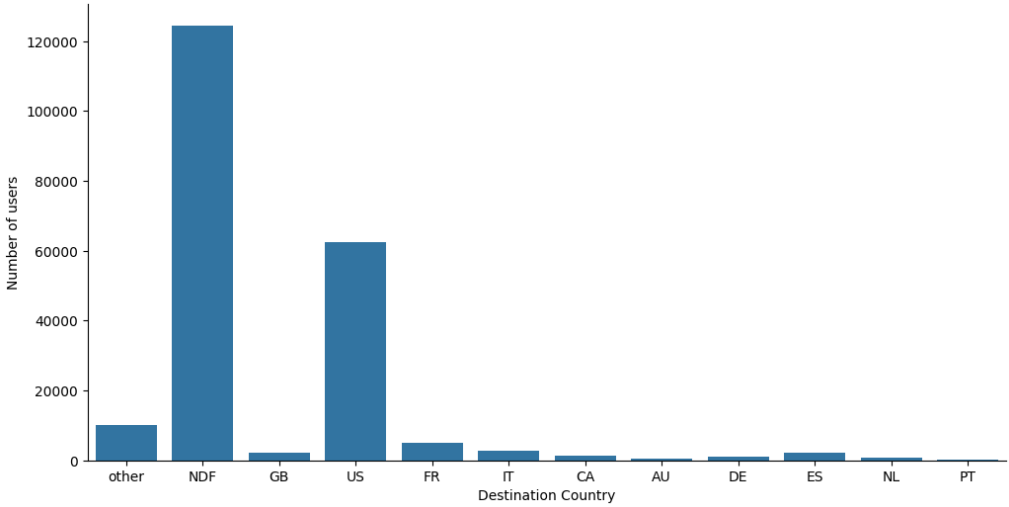


Excluding US



EDA: Destination Counts (Raw View)

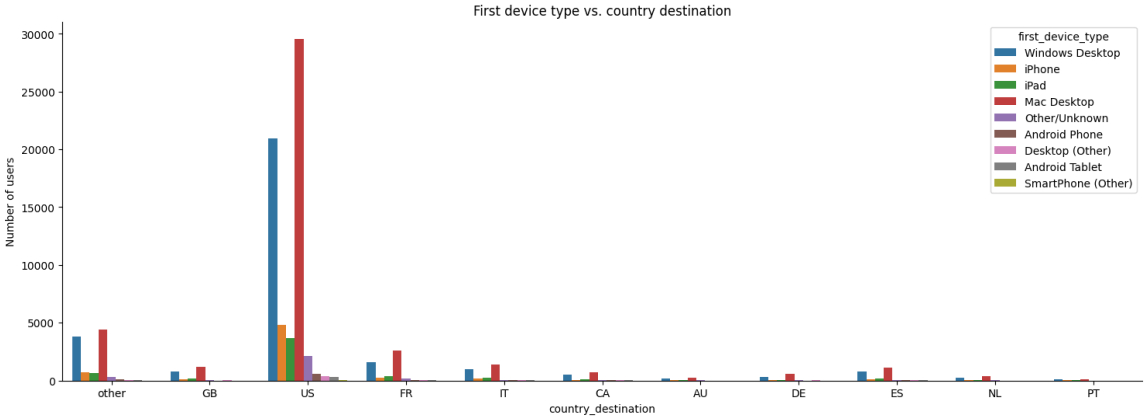
This plot shows the real count difference: `NDF` and `US` dominate the dataset.



EDA: First Device Type vs Destination

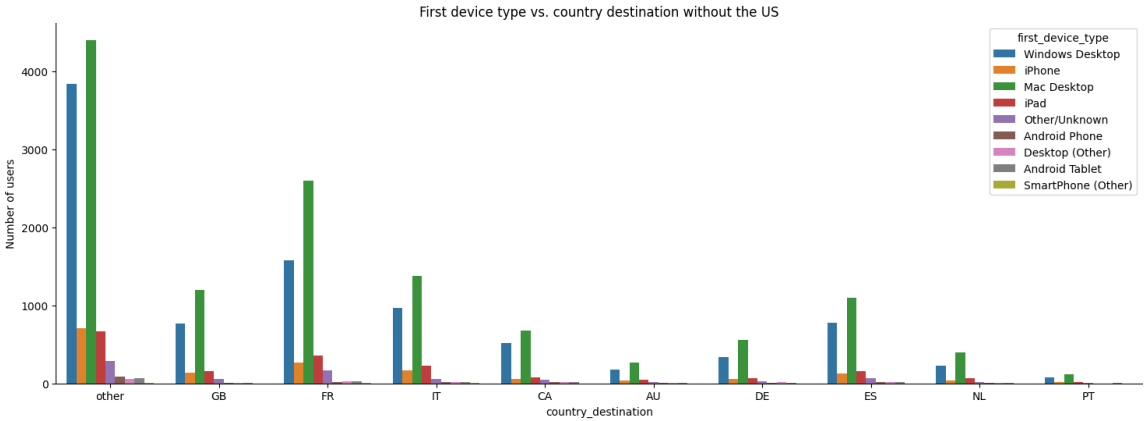
Device type shows useful signals:

- Apple devices dominate in US bookings
- For non-US destinations, Windows Desktop becomes more common



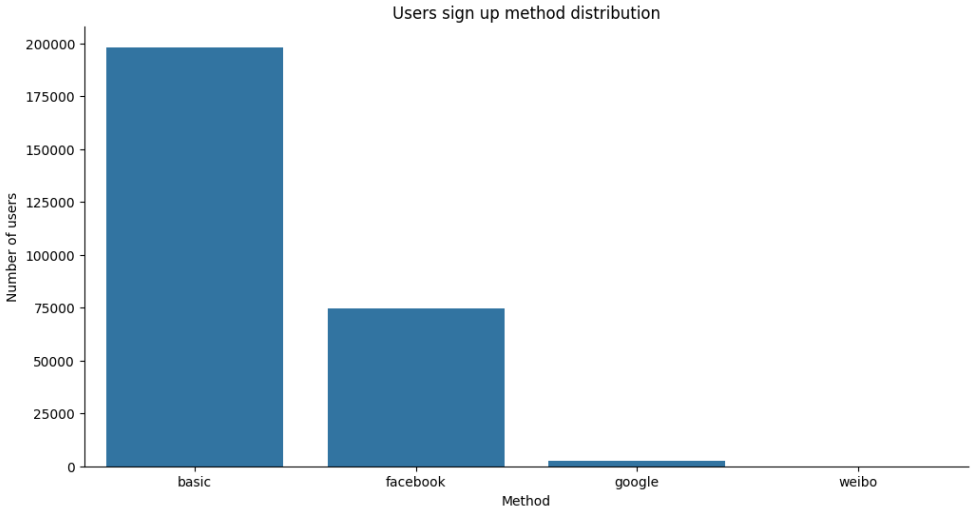
EDA: Device vs Destination (No US)

When removing US + NDF, the pattern is clearer: Europe has more Windows Desktop.



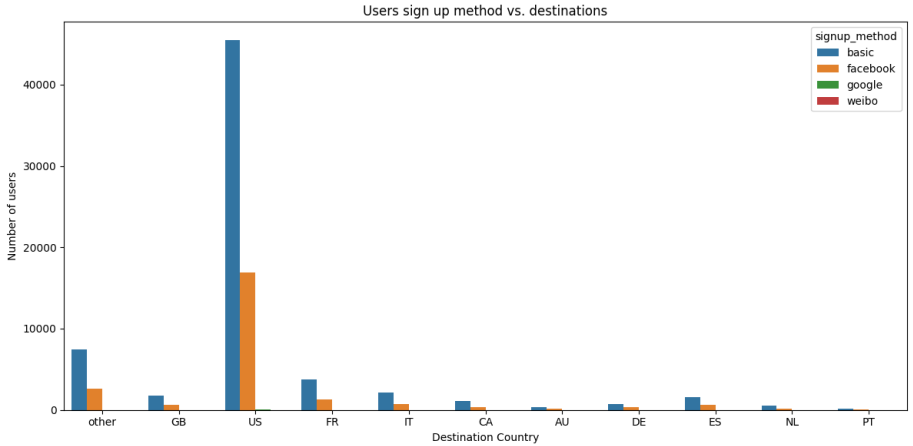
EDA: Signup Method

Most users sign up using **basic email**. Facebook is second. Google is rare.



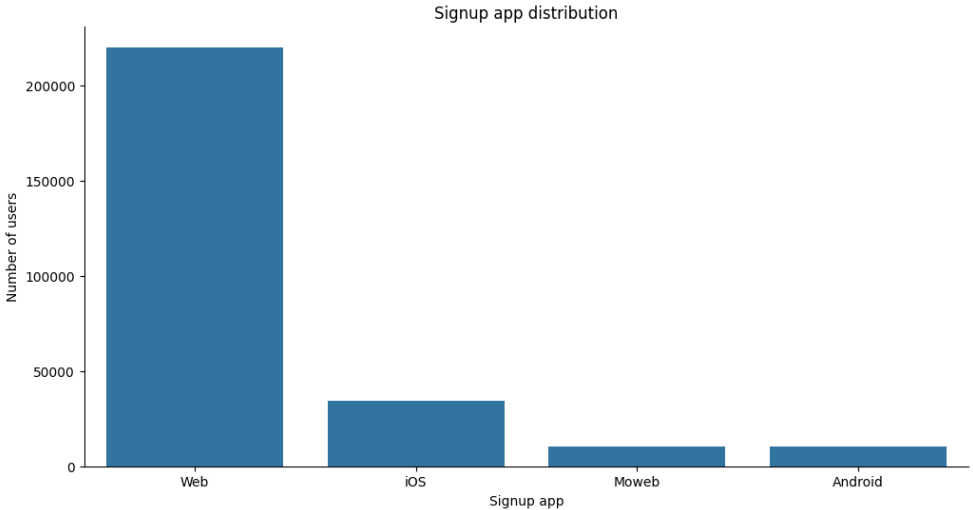
EDA: Signup Method by Destination

Basic signup dominates across destinations, so by itself it is not a strong separator, but it can still help with other features.



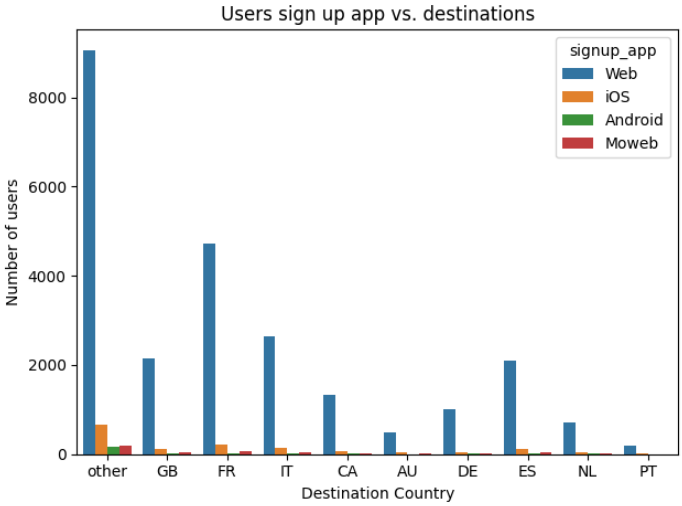
EDA: Signup App

Most users sign up via **Web**. iOS is second, Android/Moweb are smaller.



EDA: Signup App by Destination

Website is dominant for all destinations. But the iOS proportion changes across countries.



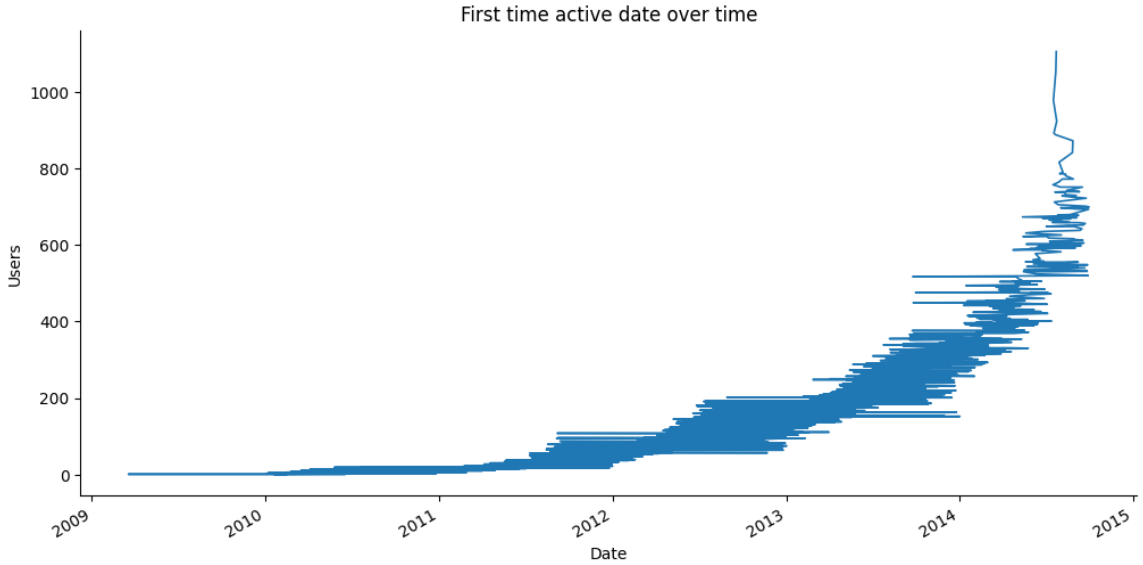
EDA: Growth Over Time

Two time signals:

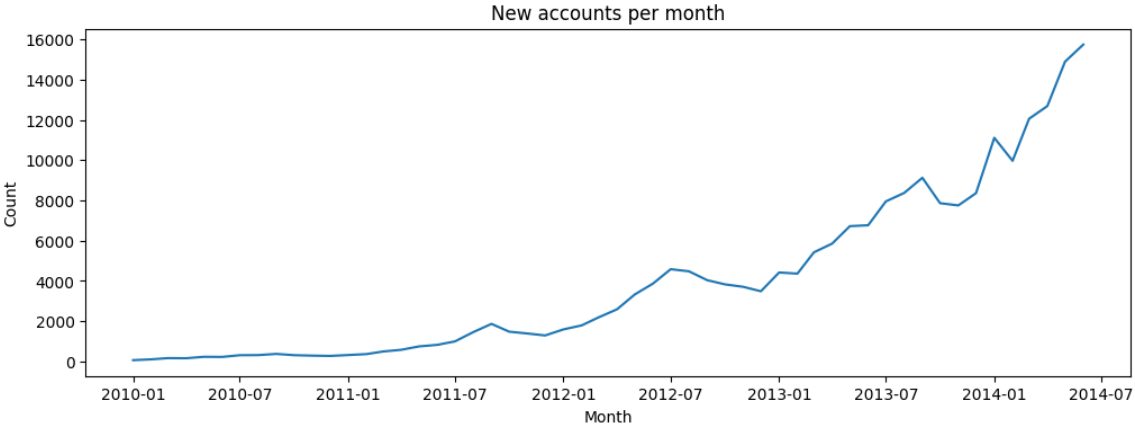
- `timestamp_first_active`
- `date_account_created`

Both show the same story: Airbnb grows fast from 2010 to 2014, especially after mid-2013.

EDA: First Active Over Time



EDA: Account Created Over Time



EDA Summary (Key Insights)

The most important things we learned:

1. Target classes are heavily imbalanced ($\texttt{NDF} + \texttt{US} \approx 87\%$)
2. Age has strong predictive value but needs careful cleaning
3. Gender has many unknowns but is still useful later
4. Device type changes with destination (Apple vs Windows patterns)
5. Sessions exist for only about half of users (missing sessions is meaningful)

Preprocessing: Why We Need It

Raw data usually contains:

- missing values
- inconsistent categories
- wrong ages
- date formats that are not ready for feature extraction

Goal: produce a **model-ready dataset with no missing values** (except destination label in test set).

Sessions Cleaning: Missing Values

Raw sessions have missing values in multiple columns:

Column	Missing values
user_id	34,496
action	79,626
action_type	1,126,204
action_detail	1,126,204
secs_elapsed	136,031

We fix them step-by-step (next slides).

Sessions Step 1: Drop Null user_id

If user_id is missing, the row cannot be linked to any user, so it is not useful.

We drop these rows:

```
sessions = sessions[sessions. user_id . notnull () ]
```

This reduces the table to about **10.53 million** rows.

Sessions Step 2–3: Fix Missing Action/Type/Detail

Step 2: Missing `action` happens mostly for message events, so we fill it with `"message"`.

Step 3: Missing `action_type` and `action_detail`:

- Pass 1: fill using the most common value for the same `action`
- Pass 2: apply special rules for a few known actions
- Remaining rare cases become `"missing"`

This makes the categorical session columns complete.

Sessions Step 4: Impute secs_elapsed

secs_elapsed is right-skewed (many small values, few huge values). So we use a **per-action median** instead of a global mean.

```
med = sessions.groupby('action')['secs_elapsed'].transform('median')
sessions['secs_elapsed'] = sessions['secs_elapsed'].fillna(med)
```

After this, sessions have **zero null values**.

Users + Sessions Merge: Why Many Session Values Are Missing

Only about **49%** of users appear in sessions. So after merging sessions onto users, about **51%** users have no sessions.

We treat this in a meaningful way:

- Session categorical columns: fill with `"missing"`
- Session numeric columns: fill with **0**

Interpretation: *No session record* means *zero actions recorded*.

Age Cleaning (Main Steps)

Age has multiple issues. We fix it using simple rules:

- If age > 1000, it is likely a birth year: age $= 2015 -$ year
- If age > 90, cap it to 90
- Missing ages are filled with median (33) **after** creating an age group feature

This keeps age useful while removing extreme noise.

Age Fix: Code Snippets

```
df_with_year = df['age'] > 1000
df.loc[df_with_year, 'age'] = 2015 - df.loc[df_with_year, 'age']

df.loc[df.age > 90, 'age'] = 90

df.age = df.age.fillna(33)
```

Date Parsing (Preparing Time Features)

We convert date columns into proper datetime format:

- `date_account_created` is already `YYYY-MM-DD`
- `timestamp_first_active` is `YYYYMMDDHHmmss` so we remove the time part and keep only the date

Then we can extract: year, month, weekday, holiday flag, business-day flag.

Date Parsing: Code Snippets

```
df['date_account_created'] = pd.to_datetime(df['date_account_created
    '])

df['timestamp_first_active'] = pd.to_datetime(
    df.timestamp_first_active // 1000000, format='%Y%m%d'
)
```

Categorical Fixes (Reduce Too Many Levels)

Some categorical columns have many rare values. If we one-hot encode them directly, the number of columns becomes too large.

So we group low-frequency categories into `"Other"`:

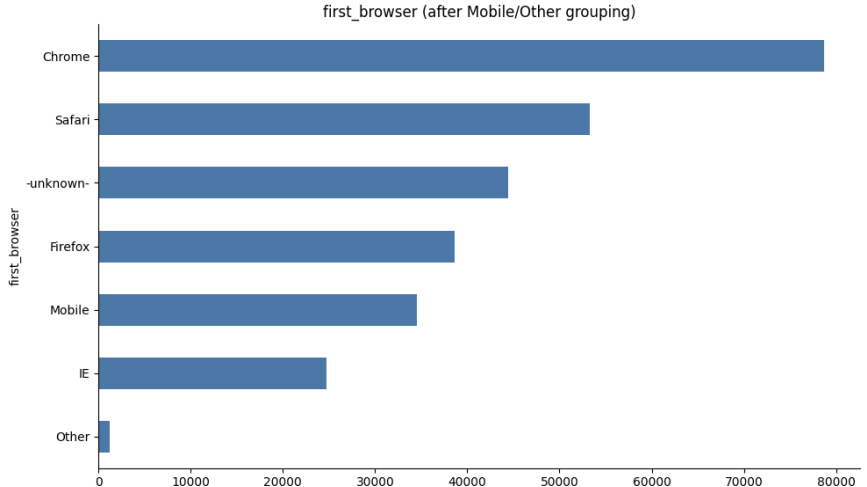
- general cut-off: 2,000 occurrences
- language cut-off: 200 occurrences (language is more important)

Also:

- `first_affiliate_tracked` missing values $\rightarrow$ `"untracked"`
- `first_browser`: consolidate mobile browsers into `"Mobile"`

First Browser Distribution (After Grouping)

After consolidation: Chrome and Safari dominate, `-unknown-` is also large, and rare browsers become `Other`.



Why Feature Engineering Matters

Sessions data is huge (10+ million rows) but it is **not** model-ready.

We need one row per user, so we aggregate sessions into a user summary.

Final goal:

- convert raw clickstream into meaningful per-user features
- combine those with demographics and signup info

Session Aggregation: Main Output

We build a new table called `sessions_new`:

- one row per user in sessions
- 23 columns (counts, modes, unique counts, time stats, device flags)

Then we merge `sessions_new` into the users table.

Feature 1: Action Count

The simplest engagement signal: **How many actions did the user do?**

```
sessions_new = (
    sessions.groupby('user_id').size()
            .reset_index(name='action_count')
)
```

High action count usually means higher engagement.

Feature 2: Most Frequent Session Values (Modes)

For each user, we take the most common value of:

- `action`
- `action_type`
- `action_detail`
- `device_type`

Simple meaning: *What is the user's typical behavior?*

Feature 3: Unique Value Counts (Behavior Diversity)

Diversity is also informative. A user who explores many pages behaves differently from a user who does only one thing.

We create:

- `unique_actions`
- `unique_action_types`
- `unique_action_details`
- `unique_device_types`

Feature 4: Duration + Pause Features

We summarize `secs_elapsed` per user:

- sum, mean, min, max, median
- pause features (day pauses, long pauses, short pauses)
- `session_length` = number of non-zero time records

Intuition:

- many day pauses can mean "thinking for days"
- many short pauses can mean "focused browsing session"

Feature 5: Device Category Flags

We create four binary flags based on the user's most common device:

- `apple_device`
- `desktop_device`
- `tablet_device`
- `mobile_device`

A device can belong to more than one group (example: Mac Desktop is Apple + Desktop).

User-Level Features: Age Group

Besides continuous age, we create `age_group` **before imputation**:

Code	Age range	Meaning
0	missing	user did not provide age
1	< 18	underage
2	18–32	young adults
3	33–42	mid-age adults
4	> 42	older adults

This preserves the information "age was missing".

One-Hot Encoding + Scaling (Preparing for Models)

Before modeling:

- We one-hot encode 13 categorical features
- We Min-Max scale continuous features into $[0, 1]$

Preprocessing is required for linear models and SVMs and generally safe for tree models.

Why Merging is Important

We have two main information sources:

- Users table (demographics + signup + marketing)
- Sessions table (behavior)

To train one model, we need **one final table** with:

- one row per user
- all features in columns

Merging Strategy: Left Join

We use a **left join** from users onto sessions summary:

- Keep **all users** (even those without sessions)
- Missing sessions are meaningful (often relates to `NDF`)

If we used an inner join, we would drop about **51%** of users and create bias.

Post-Merge: Drop Leakage and Duplicates

After merging, we drop:

- `date_first_booking` (it leaks the target: booking vs no booking)
- `user_id` (duplicate of `id`)
- merge artefact columns (like `key_0` if present)

Then we separate:

- **labels** (`country_destination`) for training rows
- **features** (all remaining columns)

Final Feature Matrix Size

After encoding + scaling, the final combined matrix has:

Property	Value
Total rows (train + test)	275,547
Total feature columns	132
Null values in features	0
Null values in labels (test only)	62,096

So each user becomes a **132-dimensional feature vector**.

Train/Validation Split (Local Evaluation)

For local experiments, we split training data:

- 80% train
- 20% validation
- using **stratification** (keeps class ratios consistent)

This is important because rare classes like `PT` are very small.

Split Code Snippet

```
from sklearn.model_selection import train_test_split
import numpy as np

y_1d = np.ravel(y)

X_train, X_val, y_train, y_val = train_test_split(
    X, y_1d,
    test_size=0.2,
    random_state=42,
    stratify=y_1d
)
```

Summary of This Part

What we completed in this "before modeling" section:

- Understood the problem and the 12-class imbalance
- Explored key patterns in demographics, devices, language, and time
- Cleaned sessions and users data carefully
- Built per-user session features (aggregation)
- Merged everything into a final $(275{,}547 \times 132)$ feature matrix

Next section : Modeling, hierarchical strategy, imbalance experiments, ensembles, and final results.

Agenda

We start directly from the **Modeling** stage and move forward:

- Evaluation metric (NDCG@5) and why it matters
- Phase 1 models: Logistic Regression, Linear SVM, Decision Tree, XGBoost, LightGBM, CatBoost
- Hierarchical modeling strategy (3-stage CatBoost)
- Imbalance handling experiments (resampling)
- Ensemble learning (weighted probability blending)
- Cross-validation (stability check)
- Final evaluation summary + feature importance insights
- Final model + discussion + future work + conclusion

Why We Focus on NDCG@5

The Kaggle competition scores predictions using **NDCG@5**. For each user, we can submit up to **5 ranked destinations**.

- If the correct destination is ranked **1st**, we get the most credit.
- If it is ranked **5th**, we get less credit.
- If it is not in the top 5, we get **0 credit**.

Key message: ranking quality matters, not just top-1 accuracy. A model must place the right class *high* in the list.

NDCG@5 Formula (Simple View)

For one user, the Discounted Cumulative Gain at rank k:

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)}$$

where $\text{rel}_i = 1$ if the prediction at rank i is correct, else 0.

Then we normalize:

$$\text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}$$

For this competition, we use **NDCG@5**.

Practical meaning: probability calibration and ranking order are crucial.

Other Metrics We Track (Supporting View)

Besides NDCG@5, we also track:

- **Accuracy**: correct top-1 prediction rate
- **Macro F1**: average F1 across classes (treats rare classes equally)
- **Weighted F1**: F1 weighted by class frequency (dominated by NDF and US)
- **Macro Recall**: average recall across classes
- **Top-5 Accuracy**: whether the true class appears anywhere in top 5

Important note: with heavy imbalance (NDF and US are huge), accuracy alone can be misleading.

Phase 1 Setup (Common for All Models)

All models were trained on:

- **Training:** 170,760 $\times$ 132 feature matrix
- **Validation:** 42,691 rows (stratified 80/20 split)
- Target has **12 classes** (NDF, US, and 10 international)

Some models require numeric labels (XGBoost, LightGBM, CatBoost), so we use a Label Encoder to map string classes to integers.

Model 1: Logistic Regression (Baseline)

Why we use it:

- Quick baseline to confirm the pipeline works
- Gives a lower-bound performance

Key setting: `class_weight="balanced"`

- Helps reduce dominance of large classes (NDF, US)
- Forces the model to pay more attention to rare destinations

Main outcome: NDCG@5 = **0.4215** (lower bound).

Logistic Regression: Configuration & Metrics

Configuration Snippet

```
from sklearn.linear_model import LogisticRegression

baseline = LogisticRegression(
    max_iter=200,
    n_jobs=-1,
    class_weight="balanced"
)
baseline.fit(X_train, y_train)
```

Interpretation: The model spreads predictions across minority classes (higher recall), but becomes imprecise for rare destinations.

Aggregate Metrics

Metric	Value
Accuracy	0.2964
Macro F1	0.0707
Weighted F1	0.3532
Macro Recall	0.1287
Top-5 Accuracy	0.5528
NDCG@5	**0.4215**

Model 2: Linear SVM

Why it helps:

- Works well with sparse one-hot features
- Scales to large datasets

Important detail: LinearSVC does not output probabilities, so we use **decision function scores** to compute NDCG@5.

Main outcome: NDCG@5 = **0.5858** (big jump over LR).

Linear SVM: Aggregate Metrics

Metric	Value
Accuracy	0.4475
Macro F1	0.0790
Weighted F1	0.4237
Macro Recall	0.1115
Top-5 Accuracy	0.7110
NDCG@5	**0.5858**

Interpretation (easy): It ranks dominant classes better and uses strong time/cohort signals, so ranking improves even without probabilities.

Logistic Regression vs. Linear SVM (Why SVM Performed Better)

- **Objective / Loss:** Logistic Regression minimizes *log-loss* (probabilistic calibration); Linear SVM minimizes *hinge loss* (margin maximization).
- **Decision Boundary:** LR fits probabilities for *all* samples; SVM focuses on *support vectors* near the boundary.
- **High-Dimensional Sparse Features:** With many one-hot features (typical in Airbnb), Linear SVM is often more robust and generalizes better.
- **Noise & Outliers:** LR is influenced by many points; SVM is less sensitive since only boundary-violating points dominate the optimization.
- **Class Imbalance:** Both can use `class_weight`, but SVM's margin-based optimization often handles rare classes more stably.

Model 3: Decision Tree

Why we include it:

- Interpretable tree baseline
- Useful comparison vs. boosted trees

We cap depth and enforce minimum leaf size to reduce overfitting: `max_depth`=20, `min_samples_leaf`=50.

Interesting pattern: Low top-1 accuracy but high top-5 coverage.

Decision Tree: Aggregate Metrics

Metric	Value
Accuracy	0.1514
Macro F1	0.0497
Weighted F1	0.2271
Macro Recall	0.1119
Top-5 Accuracy	0.7472
NDCG@5	**0.4471**

Interpretation (easy): Balanced weights make the tree explore rare-class branches, so it often puts the true class in top-5, but the ranking order is not good, so NDCG@5 stays low.

Why Decision Tree Performed Worse Than Linear Models

- **High-Dimensional Sparse Features:** With many one-hot encoded variables, single splits are weak and often noisy.
- **High Variance:** Decision Trees are unstable and sensitive to training data variations, leading to overfitting or underfitting.
- **Class Imbalance:** Trees tend to favor majority classes, reducing recall and Macro-F1 for rare destinations.
- **Overlapping Classes:** Axis-aligned splits create fragmented decision regions when classes overlap, harming generalization.
- **Single Tree Limitation:** Unlike boosting methods, a single tree cannot reduce variance through ensembling.

Model 4: XGBoost (GPU)

Why it is powerful:

- Gradient boosting captures non-linear feature interactions
- Outputs class probabilities (good for ranking)
- GPU training makes it feasible at this scale

Main outcome: NDCG@5 = **0.8301** This is a dramatic improvement over linear models.

XGBoost (GPU): Aggregate Metrics

Metric	Value
Accuracy	0.6493
Macro F1	0.1071
Weighted F1	0.6008
Macro Recall	0.1139
Top-5 Accuracy	0.9579
NDCG@5	**0.8301**

Interpretation (easy): Almost always puts the correct destination somewhere in the top 5 (Top-5 Accuracy $\approx$ 96%).

Model 5: LightGBM (GPU)

Why we test it:

- Very fast boosting library
- Leaf-wise tree growth can capture fine splits

We used `class_weight="balanced"` and multiclass objective.

Main outcome: NDCG@5 = **0.6997** Better than linear models, but noticeably worse than XGBoost/CatBoost.

LightGBM (GPU): Aggregate Metrics

Metric	Value
Accuracy	0.4760
Macro F1	0.1152
Weighted F1	0.5295
Macro Recall	0.1234
Top-5 Accuracy	0.8962
NDCG@5	**0.6997**

Interpretation (easy): Balanced weights can help fairness metrics (Macro F1/Recall), but it can distort probability ranking, hurting NDCG@5.

Model Comparison Summary (Key Characteristics & Output)

Model	Key adjectives	Strengths (our setting)	Typical output
Logistic Regression	Linear, probabilistic, interpretable, stable	Strong baseline; works with sparse data but limited without non-linearities	Class probabilities + predicted class
Linear SVM	Linear, margin-based, robust, stable	Performs well in high-dimensional sparse spaces; less sensitive to noise	Decision scores (margins) + class label
Decision Tree	Non-linear, rule-based, interpretable, high-variance	Captures simple rules; unstable and weaker on sparse/high-dim data	Class label + leaf-based probabilities
XGBoost	Ensemble, non-linear, high-performing, regularized	Models complex interactions; reduces variance; best overall performance	Class probabilities + predicted class

Model 6: CatBoost (GPU)

Why it performs well:

- Strong gradient boosting with robust training process
- Produces well-ranked probability outputs

Main outcome: NDCG@5 = **0.8304** Very close to XGBoost (0.8301).

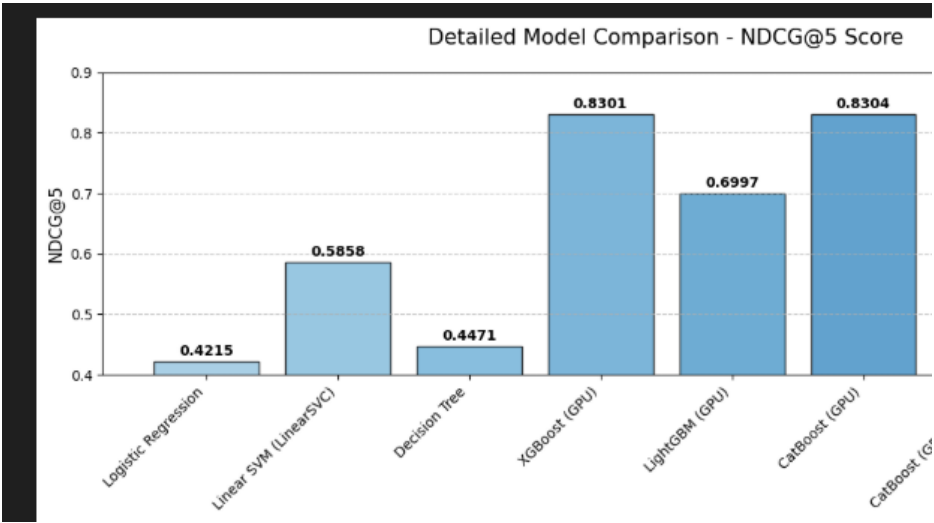
CatBoost (GPU): Aggregate Metrics

Metric	Value
Accuracy	0.6487
Macro F1	0.1068
Weighted F1	0.5998
Macro Recall	0.1136
Top-5 Accuracy	0.9585
NDCG@5	**0.8304**

Interpretation (easy): CatBoost and XGBoost are the strongest flat models. They become the base building blocks for advanced strategies.

Phase 1 Model Comparison (NDCG@5)

This plot summarizes NDCG@5 across the Phase 1 models.



Phase 1 Leaderboard (All Metrics)

Model	Acc	MacF1	WtF1	MacRec	Top5	NDCG@5
CatBoost (GPU)	0.6487	0.1068	0.5998	0.1136	0.9585	**0.8304**
XGBoost (GPU)	0.6493	0.1071	0.6008	0.1139	0.9579	0.8301
LightGBM (GPU)	0.4760	0.1152	0.5295	0.1234	0.8962	0.6997
Linear SVM	0.4475	0.0790	0.4237	0.1115	0.7110	0.5858
Decision Tree	0.1514	0.0497	0.2271	0.1119	0.7472	0.4471
Logistic Reg.	0.2964	0.0707	0.3532	0.1287	0.5528	0.4215

Boosting Models Comparison (Based on Our Results)

Model	Key characteristics	Behavior in Our Project	Typical Output
XGBoost	Gradient boosting, depth-wise tree growth, strong regularization	Best overall performance; captured complex feature interactions; stable across classes	Class probabilities + predicted class
LightGBM	Gradient boosting, leaf-wise growth, histogram-based, very fast	Faster training; competitive performance; slightly more sensitive to overfitting if not tuned	Class probabilities + predicted class
CatBoost	Gradient boosting, ordered boosting, native categorical handling	Stable performance; reduced categorical bias; slightly below XGBoost in final score	Class probabilities + predicted class

Why Go Beyond Flat 12-Class Prediction?

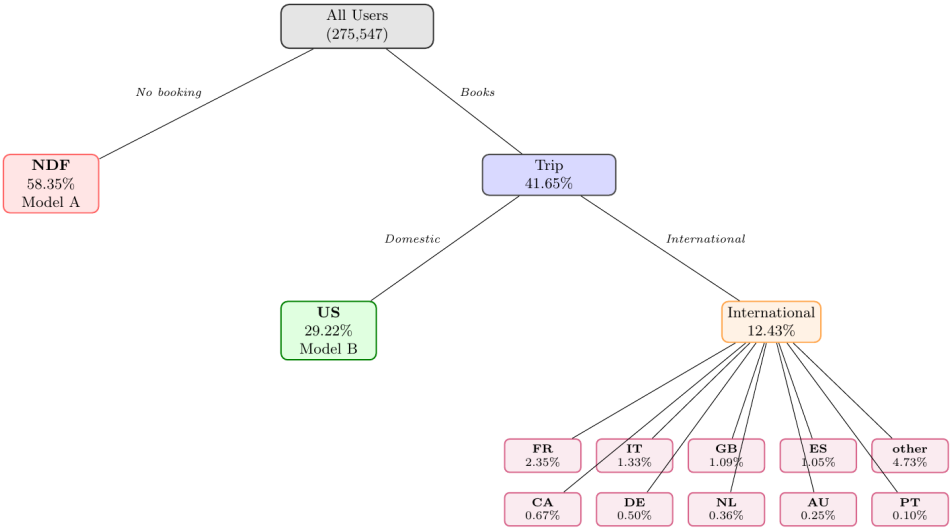
A flat model must solve **three different problems at once**:

1. Did the user book at all? (`NDF` vs others)
2. If booked, was it `US` or international?
3. If international, which country (10-way)?

Problem: class frequencies range from 58% (NDF) down to 0.10% (PT). The rare classes get weak learning signal.

Idea: split into stages so each model focuses on one simpler task.

Hierarchy Diagram (3 Levels)



Three-Stage Architecture (All CatBoost GPU)

We use CatBoost in all stages because it was one of the best flat models.

- **Stage A (Binary):** Trip vs NDF
- **Stage B (Binary):** US vs International (only on trip users)
- **Stage C (10-class):** Which international country (only on international trip users)

Why this helps (easy): Each stage gets a cleaner dataset and a cleaner target, so it can learn more accurately.

Stage A: Trip vs NDF (Target + Model)

Target:

- 1 = Trip (any label except NDF)
- 0 = NDF

Model: CatBoost binary classifier (Logloss), GPU, 600 iterations.

Output: $P(\text{Trip})$ and therefore $P(\text{NDF}) = 1 - P(\text{Trip})$.

Stage B: US vs International (Only Bookers)

Stage B only sees users who made a trip (from training truth).

- 1 = US trip
- 0 = International trip

Model: CatBoost binary classifier (Logloss), GPU, 600 iterations.

Output: $P(\text{US} \mid \text{Trip})$ and $P(\text{Intl} \mid \text{Trip}) = 1 - P(\text{US} \mid \text{Trip})$.

Stage C: Which International Country (10-Class)

Stage C only sees users with an international trip (not US).

Model: CatBoost multi-class classifier (MultiClass), GPU, 800 iterations.

Output: a probability vector over the 10 international classes: $\mathbf{p}_C \in \mathbb{R}^{10}$.

How We Combine Probabilities (Chain Rule)

We build final probabilities over all 12 classes.

$$P(\texttt{NDF}) = 1 - P(\text{Trip})$$

$$P(\texttt{US}) = P(\text{Trip}) \cdot P(\text{US} \mid \text{Trip})$$

$$P(c_k) = P(\text{Trip}) \cdot (1 - P(\text{US} \mid \text{Trip})) \cdot P(c_k \mid \text{Trip, Intl})$$

Simple meaning: probability is "routed" step-by-step through the hierarchy.

Hierarchical Model Results (Big Win)

Metric	Flat CatBoost	Hierarchical CatBoost
Accuracy	0.6487	**0.7307**
Macro F1	0.1068	**0.1199**
Weighted F1	0.5998	**0.6670**
Macro Recall	0.1136	**0.1253**
Top-5 Accuracy	0.9585	**0.9591**
NDCG@5	0.8304	**0.8610**

Key message: it improves *every single metric*, especially accuracy and NDCG@5.

Why Imbalance is a Big Issue

The largest class is `NDF` (58.35%) and the smallest is `PT` (0.10%). That is roughly a **583:1** ratio.

This causes:

- Majority-class dominance (models get high accuracy by focusing on NDF/US)
- Low Macro F1/Recall (rare countries are predicted poorly)
- Weak learning signal ("gradient starvation") for rare classes

Two Resampling Strategies We Tested

We tested two approaches using CatBoost as the base learner:

1. **Hybrid Sampling:** RandomUnderSampler (reduce NDF/US) + SMOTE (grow minorities)
2. **Balanced Bootstrap:** sample equal number per class (with replacement)

Important: We always evaluate on the original validation set (no resampling there), so comparisons stay fair.

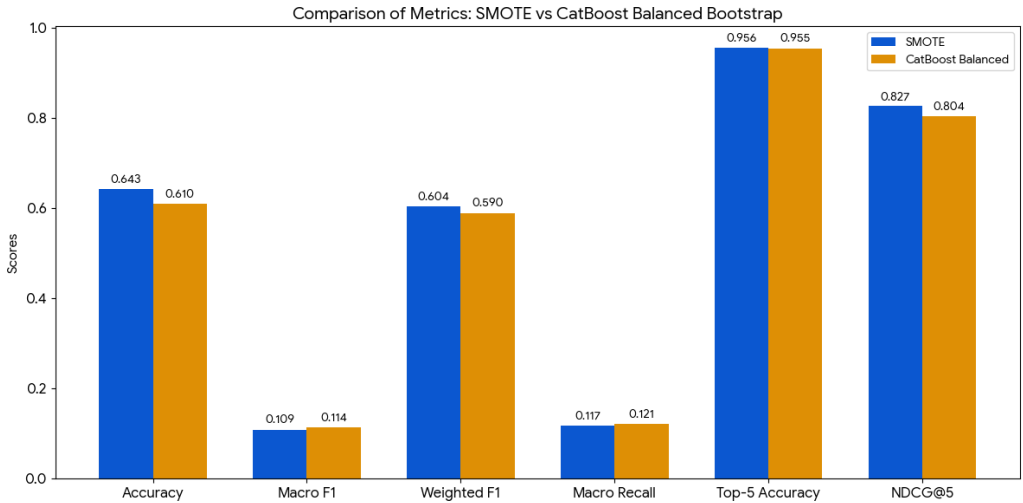
Comparison of Sampling Strategies

Metric	Hybrid Sampling	Balanced Bootstrap
Accuracy	0.6429	0.6104
Macro F1	0.1089	0.1138
Weighted F1	0.6042	0.5899
Macro Recall	0.1174	0.1207
Top-5 Accuracy	0.9562	0.9554
NDCG@5	**0.8269**	**0.8041**

Hybrid Sampling
Slightly better fairness, but NDCG@5 drops vs. baseline.

Balanced Bootstrap
Best fairness improvement, but biggest NDCG@5 loss.

Imbalance Strategies: NDCG@5 Comparison Plot



Main conclusion: Resampling tends to hurt the competition metric (NDCG@5).

Decision: No Resampling in Final Pipeline

Why resampling hurts NDCG@5 (easy explanation):

- NDF + US together are **87.6%** of users
- NDCG@5 rewards confident correct ranking
- Resampling reduces confidence on majority classes
- So ranking quality decreases on most users

Better solution: hierarchical modeling improves both fairness and NDCG@5 without changing the data distribution.

Why Ensemble Models?

Ensembles combine multiple models to reduce errors.

Simple idea: If two models make different mistakes, averaging them can cancel some errors.

We ensemble:

- Hierarchical CatBoost (structurally different)
- Flat CatBoost (Optuna tuned)
- (Optionally) XGBoost

Optuna Tuning (CatBoost)

We used Optuna (20 trials) to tune CatBoost for NDCG@5.

Best found settings:

Parameter	Best Value
iterations	705
learning_rate	0.08635
depth	6
l2_leaf_reg	2.6546
random_strength	1.2401
bagging_temperature	0.1310

It improves only slightly over default CatBoost (very small gain), but it is useful as an independent ensemble member.

2-Model Ensemble (Hierarchical + CatBoost Optuna)

We blend probabilities with weights:

$$\text{Ensemble} = \frac{0.7 \cdot P_{\text{Hier}} + 0.3 \cdot P_{\text{CatOpt}}}{2}$$

The division by 2 does **not** change ranking (just scales values).

Result: NDCG@5 improves from 0.8610 to **0.8651**.

2-Model Ensemble: Metrics

Metric	Hierarchical Only	2-Model Ensemble
Accuracy	0.7307	**0.7422**
Macro F1	0.1199	**0.1233**
Weighted F1	0.6670	**0.6817**
Macro Recall	0.1253	**0.1291**
Top-5 Accuracy	0.9591	0.9586
NDCG@5	0.8610	**0.8651**

3-Model Ensemble (Add XGBoost)

We also tried a 3-model blend:

$$P = 0.60 \cdot P_{\text{Hier}} + 0.15 \cdot P_{\text{Cat}} + 0.25 \cdot P_{\text{XGB}}$$

But it **reduced** NDCG@5 to 0.8595.

Easy reason: XGBoost is weaker than the hierarchical model, so adding it with a large weight can "pull down" the final ranking quality.

Why We Do Cross-Validation

A single 80/20 split might be “lucky” or “unlucky”.

Stratified 5-fold CV helps by:

- keeping class proportions similar in each fold
- producing a more reliable estimate of generalization
- measuring performance stability (variance)

5-Fold CV Results (Simplified Ensemble)

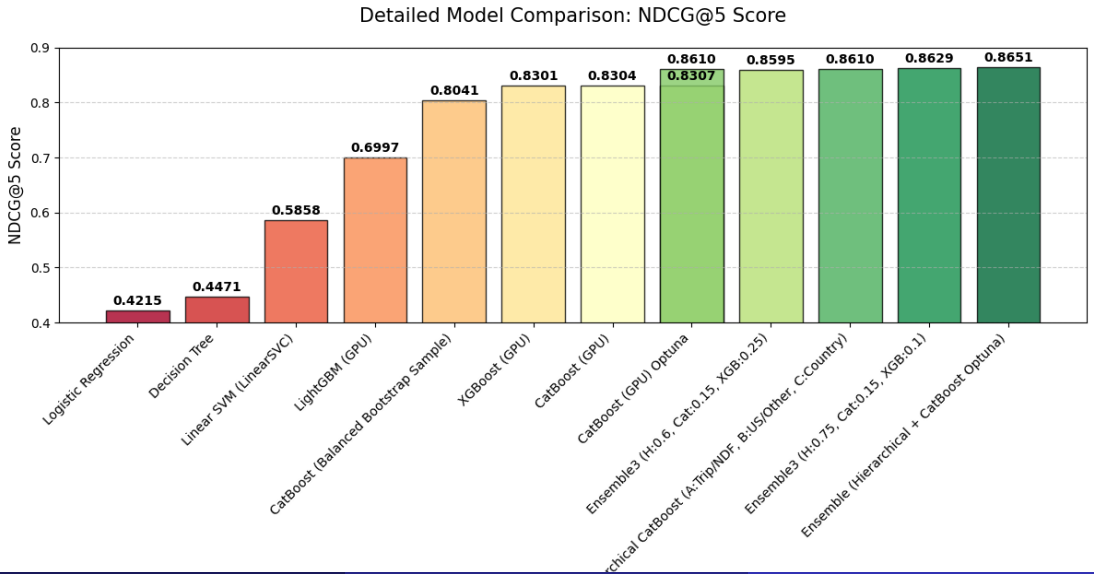
Fold	NDCG@5
Fold 1	0.8303
Fold 2	0.8302
Fold 3	0.8305
Fold 4	0.8306
Fold 5	0.8308
Mean	**0.8305**
Std	**0.0002**

Key message: extremely stable performance across folds.

Model Performance Leaderboard

#	Model	Acc.	Macro F1	Wt. F1	Mac. Rec.	Top-5	NDCG@5
1	**Ensemble 2-Model**	**0.7422**	**0.1233**	**0.6817**	0.1291	0.9586	**0.8651**
2	Hierarchical CatBoost	0.7307	0.1199	0.6670	0.1253	0.9591	0.8610
3	Ensemble 3-Model	0.7272	0.1201	0.6668	0.1260	0.9587	0.8595
4	CatBoost (Optuna)	0.6490	0.1069	0.6001	0.1136	0.9591	0.8307
5	CatBoost (Default)	0.6487	0.1068	0.5998	0.1136	0.9585	0.8304
6	XGBoost (GPU)	0.6493	0.1071	0.6008	0.1139	0.9579	0.8301
7	CatBoost (Hybrid)	0.6429	0.1089	0.6042	0.1174	0.9562	0.8269
8	CatBoost (Balanced)	0.6104	0.1138	0.5899	0.1207	0.9554	0.8041
9	LightGBM (GPU)	0.4760	0.1152	0.5295	**0.1234**	0.8962	0.6997
10	Linear SVM	0.4475	0.0790	0.4237	0.1115	0.7110	0.5858
11	Decision Tree	0.1514	0.0497	0.2271	0.1119	0.7472	0.4471
12	Logistic Regression	0.2964	0.0707	0.3532	0.1115	0.5528	0.4215

Progression of NDCG@5 Through Development



Three Performance Tiers (Easy Summary)

We observe 3 clear tiers:

- **Tier 1 (0.42–0.59):** Logistic Regression, Decision Tree, Linear SVM
- **Tier 2 (0.70–0.83):** boosted flat models (CatBoost/XGBoost/LightGBM + resampling)
- **Tier 3 (0.86+):** hierarchical and ensemble models

Main takeaway: Architecture decisions matter much more than small hyperparameter tuning.

Why Feature Importance Matters

Understanding important features helps us:

- verify the model learned real signals (not noise)
- explain results to non-technical audiences
- guide future feature engineering

We use two views:

- **Gain-based** importance (from boosted trees)
- **Permutation** importance (model-agnostic, validation-based)

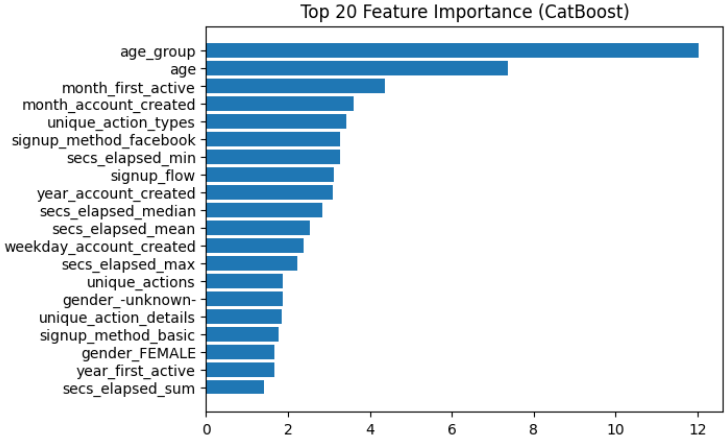
Most Consistent Important Features (Across Models)

Across almost all models, the strongest signals are:

- **Age / Age group** (strongest single predictor)
- **Temporal registration** features (month/year/weekday first active, account created)
- **Session engagement** stats (secs_elapsed_median/mean/min/max, unique action types)
- **Signup method** (facebook vs basic)

Simple story: Who the user is (age), when they joined (time), and how they behave (sessions) are the best signals for destination prediction.

Frame Title


Top 20 Feature Importance (CatBoost)
age_group
age
month_first_active
month_account_created
unique_action_types
signup_method_facebook
secs_elapsed_min
signup_flow
year_account_created
secs_elapsed_median
secs_elapsed_mean
weekday_account_created
secs_elapsed_max
unique_actions
gender_-unknown-
unique_action_details
signup_method_basic
gender_FEMALE
year_first_active
secs_elapsed_sum
0
2
4
6
8
10
12

Stage C (International Predictor) Feature Importance (Top Examples)

In the hierarchical model, Stage C is trained only on international travelers, so it highlights slightly different signals.

Top features include:

- age, `month_first_active`, `weekday_account_created`
- session duration stats like `secs_elapsed_median/max/min`
- gender signals become more visible (male/female both important)

Easy meaning: Among international travelers, demographic differences and behavior patterns become clearer because the training set is more focused.

Selected Final Pipeline

Final model: 2-model weighted probability ensemble

- Hierarchical CatBoost (3-stage)
- CatBoost Optuna-tuned (flat)

Weights: 0.70 (Hierarchical) / 0.30 (Optuna)

Why we choose it:

- Best NDCG@5 = **0.8651**
- Improves almost every metric
- Strong stability evidence from cross-validation

Final Model: Performance Table

Metric	Flat CatBoost	Hierarchical	Final Ensemble
Accuracy	0.6487	0.7307	**0.7422**
Macro F1	0.1068	0.1199	**0.1233**
Weighted F1	0.5998	0.6670	**0.6817**
Macro Recall	0.1136	0.1253	**0.1291**
Top-5 Accuracy	0.9585	0.9591	0.9586
NDCG@5	0.8304	0.8610	**0.8651**

Discussion (Main Lessons)

Main lessons from the project:

- **Problem decomposition beats model complexity:** hierarchy gave a much larger gain than tuning.
- **Boosted trees are essential:** linear/shallow models cannot capture the interactions.
- **Resampling is not ideal for NDCG@5:** it can hurt ranking quality on the majority of users.
- **Age is the strongest universal feature:** appears in top importance across models.

Future Work (Practical Ideas)

Possible improvements:

- Better age representation (more bins, splines, interactions)
- Sequence-aware session modeling (RNN/Transformer over action sequence)
- Probability calibration (Platt scaling / isotonic regression)
- Improve Stage C with models designed for low-data multi-class settings
- Add richer time features (holiday/business-day signals, month$\times$year interactions)

Conclusion

We moved from a simple baseline (NDCG@5 = 0.4215) to a strong final pipeline (NDCG@5 = **0.8651**) by:

- switching from linear models to gradient boosting
- exploiting the natural hierarchy (book / US vs intl / which country)
- applying a small but effective ensemble blend
- verifying stability using cross-validation

Final takeaway: understanding the structure of the problem was the biggest win.

Questions?